

Guía de Refactorización y Configuración en GitHub Projects

Basado en la estricta devolución del profesor, he estructurado un plan de acción en cascada (paso a paso) para dejar el proyecto impecable antes de arrancar con los microservicios.

Configuración inicial en GitHub

Antes de cargar las tareas, configura tu entorno en GitHub para que tus compañeros entiendan cómo trabajar:

1. Crear el Milestone (Hito):

- Ve a la pestaña **Issues** -> **Milestones** -> **New milestone**.
- **Título:** Fase 1.5 - Refactorización y Correcciones Arquitectónicas.
- **Descripción:** Solucionar todas las violaciones de capas, principios DRY/KISS/YAGNI y reparar la suite de testing según la devolución del profesor, dejando el proyecto listo para microservicios.

2. Crear Etiquetas (Labels):

- Ve a **Issues** -> **Labels** -> **New label**. Crea las siguientes para medir prioridad y dificultad:
 -  Prioridad: Alta (Bloqueante, se debe hacer ya)
 -  Prioridad: Media (Importante para la nota)
 -  Prioridad: Baja (Limpieza técnica)
 -  Dificultad: Alta
 -  Dificultad: Media
 -  Dificultad: Baja
 -  Arquitectura /  Testing /  Clean Code

Tickets / Tareas (Orden en Cascada)

Deberás crear un "Issue" en GitHub por cada uno de estos ítems, asignarle el Milestone creado, y arrastrarlos a la columna "**Todo**" de tu tablero. El orden está pensado para que **NO se pisen entre ustedes**. Por ejemplo, no pueden refactorizar la arquitectura (Item 3) si los tests siguen rotos (Item 1).

 Fase 1: Estabilizar el Proyecto (TDD)

Estas tareas deben hacerse primero para asegurar que nada se rompa en los siguientes pasos.

Ítem 1: Reparar tests rotos actuales

- **Prioridad:** 🔴 Alta | **Dificultad:** 🥗 Baja | **Label:** 🧪 Testing
- **Descripción:** La suite de tests actual falla porque el endpoint /upload devuelve un mensaje distinto al que espera el test, y porque al enviar un string falso (en vez de un PDF real), pdfplumber lanza un error 500.
- **Pasos a seguir (Sub-tareas):**
 - Reemplazar b"Texto simulado..." en los tests por la carga de un archivo PDF real pero mínimo (ej. dummy.pdf de 1 pág).
 - Actualizar las aserciones (assert) en test_main.py para que coincidan con los nuevos mensajes de éxito de routers.py.

Ítem 2: Aislar tests e implementar Mocks (Reglas F.I.R.S.T)

- **Prioridad:** 🔴 Alta | **Dificultad:** 🌶️ Alta | **Label:** 🧪 Testing
- **Descripción:** Los tests son lentos y dependientes del orden. Fallan si no hay .env. Hay que independizarlos y evitar que golpeen a la IA o a la BD real durante el testeo.
- **Pasos a seguir:**
 - Eliminar la variable global doc_id_prueba. Usar fixtures de Pytest para que cada test cree y borre su propia data aisladamente.
 - Instalar pytest-mock y mockear cliente_ia.generate y las llamadas a MongoDB en los tests.
 - Crear un archivo .env.example en la raíz con valores vacíos para que los tests pasen en clones limpios.

🏗️ Fase 2: Refactorización Arquitectónica

Una vez que los tests pasan en verde, se puede modificar el código con seguridad.

Ítem 3: Crear entidad de Dominio Documento

- **Prioridad:** 🟡 Media | **Dificultad:** 🥗 Baja | **Label:** 🏗️ Arquitectura

- **Descripción:** El profesor observó que la carpeta `app/models/` está vacía y pasamos diccionarios crudos.
- **Pasos a seguir:**
 - Crear archivo `documento.py` en `app/models/`.
 - Crear una clase `Documento` (usando `Pydantic` o `Dataclasses`) que defina los campos del PDF (`id`, `filename`, `text`, `resumen`, `checksum`).

Ítem 4: Extraer lógica del Router al Service (Eliminar Controlador Gordo)

- **Prioridad:** 🔴 Alta | **Dificultad:** 🍌 Media | **Label:** 🏗️ Arquitectura
- **Descripción:** El endpoint `POST /upload` orquesta toda la aplicación (validación, IA, BD), rompiendo la arquitectura en capas. Debe ser "tonto" y delegar todo al Service.
- **Pasos a seguir:**
 - En `service/pdf_service.py`, crear la función `procesar_archivo(contenido, filename, content_type) -> Documento`.
 - Mover toda la lógica de validación, extracción, IA y guardado a esta nueva función.
 - En `routers.py`, el endpoint `/upload` solo debe llamar a `pdf_service.procesar_archivo` y devolver el resultado.

Ítem 5: Eliminar dependencias entre Router y Repositorio

- **Prioridad:** 🔴 Alta | **Dificultad:** 🥗 Baja | **Label:** 🏗️ Arquitectura
- **Descripción:** La capa de presentación no debe saltarse la capa de negocio. Actualmente `routers.py` importa `db_repo.py` directamente.
- **Pasos a seguir:**
 - En `service/pdf_service.py`, crear funciones intermedias (wrapper) como `obtener_documento_por_id`, `listar_documentos`, `borrar_documento`.
 - En `routers.py`, reemplazar todos los imports de `repository.db_repo` por los de `service.pdf_service`.

🔪 Fase 3: Clean Code y Alineación

Limpieza general de las observaciones de principios de diseño.

Ítem 6: Resolver violaciones al principio DRY y YAGNI

- **Prioridad:** 🟡 Media | **Dificultad:** 🥰 Baja | **Label:** 🛠️ Clean Code
- **Descripción:** Se repite código de serialización, hay dependencias sin uso y configuraciones duplicadas.
- **Pasos a seguir:**
 - En db_repo.py: Crear una función `_serializar(doc)` para centralizar la conversión de `ObjectId` a `str`.
 - En db_repo.py: Eliminar el bloque de fallback que busca por string si falla el `ObjectId`. Solo atrapar `InvalidId` y retornar `None`.
 - En pdf_service.py: Eliminar `os.getenv` y usar `settings.ollama_url` importando las configuraciones.
 - En pyproject.toml: Desinstalar y quitar `pypdf2` (ya que se usa `pdfplumber`).

Ítem 7: Alinear Documentación UML

- **Prioridad:** 🟢 Baja | **Dificultad:** 🥰 Baja | **Label:** ⚙️ Arquitectura
- **Descripción:** Los archivos `.puml` declaran clases (como `Validador` o `Repository` orientado a objetos) que no existen en el código real de Python.
- **Pasos a seguir:**
 - Actualizar `docs/clases.puml` y `docs/secuencia.puml` para que reflejen las funciones y módulos reales que se están usando en el código tras refactorizar (o crear las clases orientadas a objetos si deciden implementar el patrón `Repository` estrictamente).